

Architectures des ordinateurs

Langage machine :
assembleur 8086

Pr.SADIQ MOUNIR

Plan



Processeur 8086



Assembleur

Caractéristiques du 8086

- ⇒ Bus de données : 16 bits
- ⇒ Bus d'adresse : 20 bits
- ⇒ Registres : 16 bits

⇒ 4 accumulateurs 16 bits

- ⇒ Accumulateur (AX)
- ⇒ Base (BX)
- ⇒ Counter (CX)
- ⇒ Accumulateur auxiliaire (DX)

AH	AL
BH	BL
CH	CL
DH	DL

Certains registres
16 bits peuvent être
utilisés comme deux
registres 8 bits

⇒ Registres accessibles sous forme de 2 info 8 bits

- ⇒ AX se décompose en AH (poids fort) et AL (poids faible de AX)...

Caractéristiques du 8086

⇒ 4 accumulateurs 16 bits

⇒ AX, BX, CX, DX

⇒ Registres d'index et de pointeur :

⇒ Pointeur d'instruction (IP)

⇒ Index source ou destination (SI, DI)

⇒ Pointeur de Pile ou de base (SP, BP)

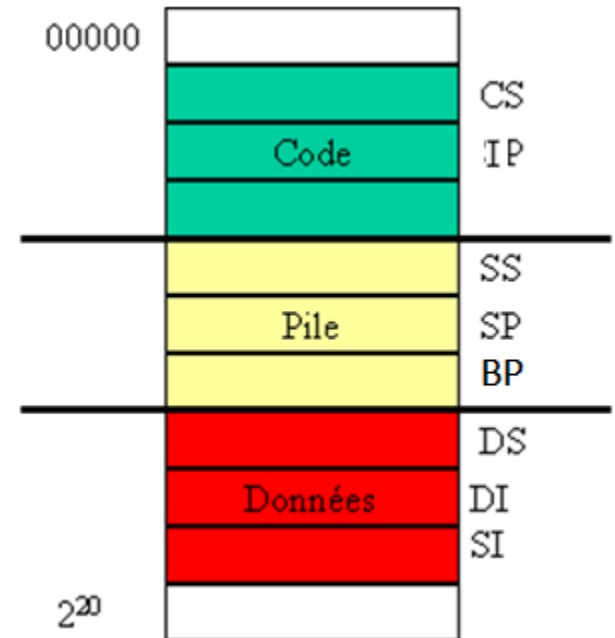
⇒ 3+1 registres segment :

⇒ Segment de code (CS) : contient le prog en cours d'exécution

⇒ Segment data (DS) : contient les données du programme

⇒ Segment stack (SS) : contient des données particulières

⇒ Extra Segment (ES)



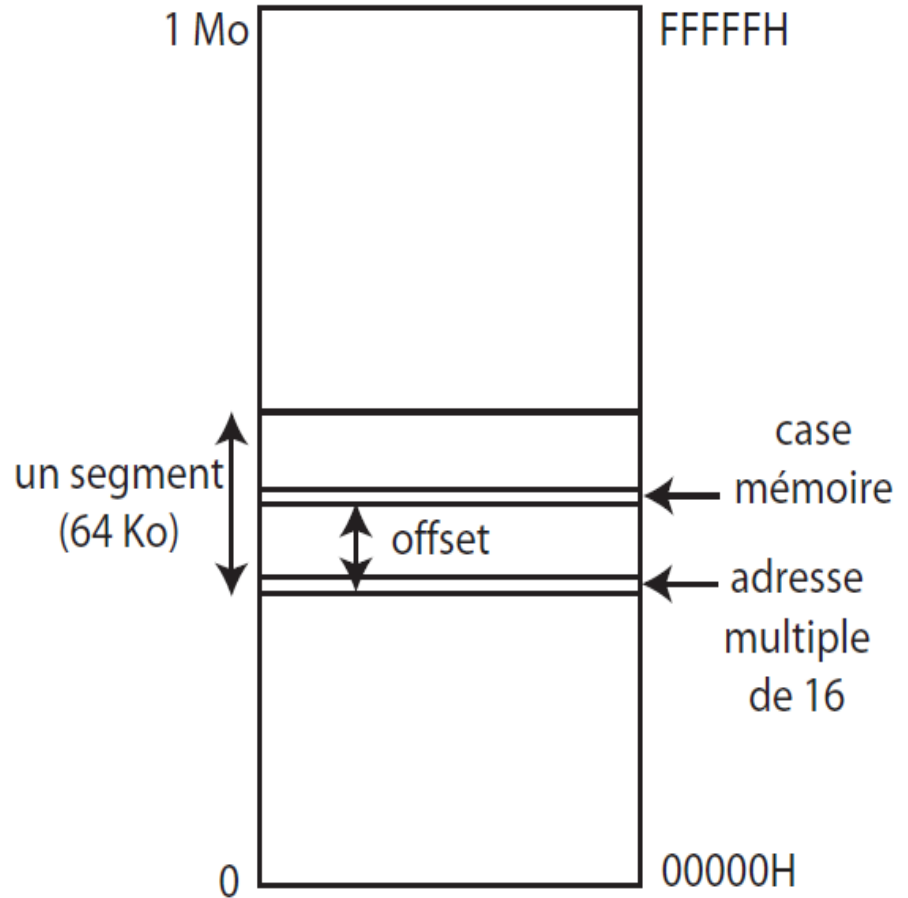
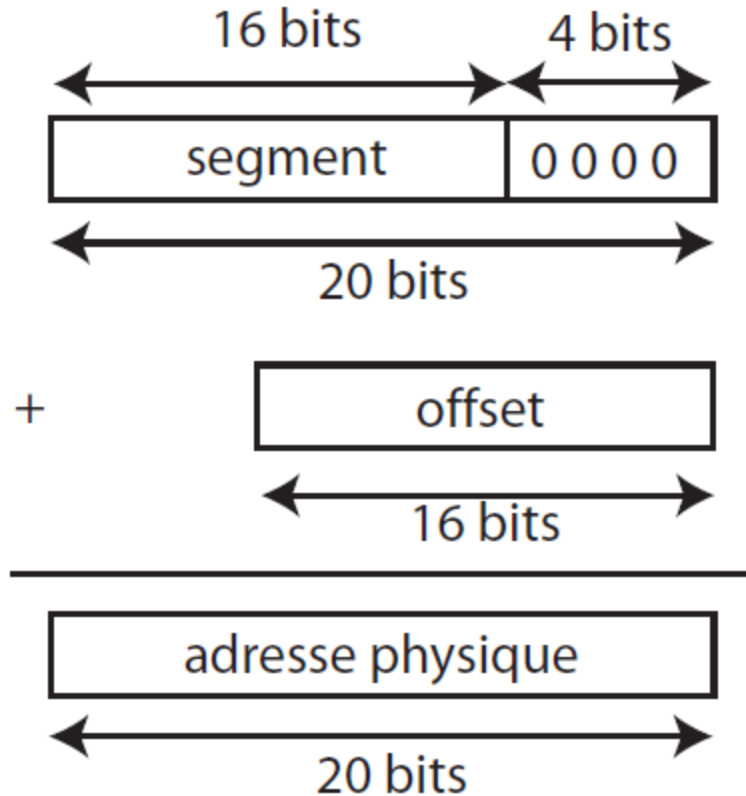
Segmentation de la mémoire

- ⇒ Largeur du bus d'adresse = 20 bits
 - ⇒ Possibilité d'adressage mémoire = $2^{20} = 1 \text{ Mo}$
- ⇒ Le pointeur d'instruction fait 16 bits
 - ⇒ Possibilité d'adresser $2^{16} = 64 \text{ Ko}$ (ce qui ne couvre pas la mémoire)
- ⇒ On utilise deux registres pour indiquer une adresse au processeur
 - ⇒ Chaque segment débute à l'endroit spécifié par un registre spécial nommé registre segment.
 - ⇒ Le déplacement permet de trouver une information à l'intérieur du segment.
 - ⇒ CS:IP : lecture du code d'une instruction (CS registre segment et IP déplacement)
 - ⇒ DS : accès aux données (MOV AX,[1045] = lecture du mot mémoire d'adresse DS:1045H)

Segmentation de la mémoire

- ⇒ Registres de déplacement = sélectionner une information dans un segment.
- ⇒ Dans le segment de code CS : le compteur de programme IP joue ce rôle. CS:IP permet d'accéder à une information dans le segment de code.
- ⇒ Dans les segments de DS : les deux index SI ou DI jouent ce rôle. Le déplacement peut être aussi une constante. DS:SI ou DS:DI permettent d'accéder à une information dans le segment de données.
- ⇒ Dans le segment de pile SS le registre SP (stack pointer) et BP (base pointer) jouent ce rôle. SS:SP ou SS:BP permettent d'accéder à une information dans le segment de pile.

Adresse logique et Adresse physique :



Ainsi, l'adresse physique se calcule par l'expression :

$$\text{adresse physique} = 16 \times \text{segment} + \text{offset}$$

Architecture des ordinateurs
Série 3

Exercice 1 (partie1 et 2)

Exercice 6

Accès à la mémoire

- Si **adr** est une adresse mémoire, alors **[adr]** représente le contenu de l'adresse **adr**
- **adr** peut être :
 1. une constante ou valeur immédiate (p. ex. [123]),
 2. une étiquette (p. ex. [var]),
 3. un registre (p. ex. [ax]),
 4. une expression limitée à certains opérateurs (p. ex. [2*eax+var+1]).

Instructions

❑ Instructions de transfert : registres ↔ mémoire

- Copie : **mov**
- Gestion de la pile : push, pop

❑ Instructions de calcul

- Arithmétique : add, sub, mul, div
- Logique : and, or
- Comparaison : cmp

❑ Instructions de saut

- sauts inconditionnels : jmp
- sauts conditionnels : je, jne, jg, jl, jle, jge
- appel et retour de procédure : call, ret

Copie - mov

- **Syntaxe :**

mov destination source

- Copie **source** vers **destination**
- **source** : un registre, une adresse ou une constante
- **destination** : un registre ou une adresse
- Les copies registre - registre sont possibles, mais pas les copies mémoire – mémoire

- **Exemples :**

mov ax, bx	; reg reg
mov ax, [var]	; reg mem
mov bx, 12	; reg constante
mov [var], ax	; mem reg

Nombre d'octets copiés

- Lorsqu'on copie vers un registre ou depuis un registre , c'est la taille du registre qui indique le nombre d'octets copiés
- Lorsqu'on copie une constante en mémoire, il faut préciser le nombre d'octets à copier, à l'aide des mots clefs
 - byte un octet
 - word deux octets
 - dword quatre octets

- Exemples :

```
mov ax, bx           ; reg reg
Mov ax, [var]         ; reg mem
Mov bx, 12            ; reg constante
mov [var], ax         ; mem reg
mov dword [var], 1    ; mem constante
```